Архитектурный анализ: AI-native система учёта

> **Фокус:** Не только ГСМ — оценка архитектурного подхода к учёту в бизнесе в эпоху AI-агентов.
> **Дата:** 2026-02-27 (обновлено)
> **Автор:** Claude Opus 4.6 (по запросу МАГ)
>
> **Маркетинговая концепция (2026-02-27):** ClearLedger — буферная зона между реальным миром и 1С. Три кейса ценности: (1) приём + верификация с эталоном из 1С и внешними сервисами, (2) AI-анализ незакрытого периода с непрерывными сверками, (3) моделирование закрытия периода и рекомендации по оптимизации. Подробности → PLAN.md, раздел «Маркетинговая концепция».
>
> **См. также:** LAYER2_ARCHITECTURE.md — зафиксированное архитектурное решение (ADR) от 2026-02-26: трёхуровневая архитектура данных (Layer 1 RAW + Layer 1a Staging + Layer 2 Public), модель поставки (Docker), обработка на клиенте и в облаке, SQL-схемы.

---

1. СТРАТЕГИЧЕСКАЯ ОЦЕНКА: Правильный ли это подход?

1.1. Короткий ответ

Да, это правильный подход. Трёхслойная архитектура (RAW → Normalized → Clean → 1С) — это по сути **accounting middleware**, которое решает фундаментальную проблему: традиционные системы учёта (1С, SAP) проектировались для ручного ввода, а не для автоматической обработки разнородных данных с участием AI.

Но есть **4 принципиальные коррекции**, которые превратят хорошую архитектуру в отличную.

1.2. Почему этот подход верный

Проблема, которую вы решаете:

```
БЫЛО (классика):                    СТАЛО (ваш подход):

Данные → Бухгалтер → 1С             Данные → Система → AI + Человек → 1С
         (ручной ввод,                       (автопарсинг,   (сверка,
```

```
(только
          ошибки,                                нормализация)   аналитика)
печать)
          задержки)
```

Вы по сути строите **"мозг" перед "принтером"**. Это абсолютно правильно, потому что:

1. **AI-агенты работают с данными, не с интерфейсами.** Неважно, насколько удобен UI 1С — для AI нужна машиночитаемая структура данных с API. Ваш слой 2 (Normalized) — это именно это.
2. **Иммутабельность = доверие.** В учёте нельзя «потерять» данные. Ваш append-only RAW слой + audit trail корректировок — это правильный паттерн для любого типа учёта, не только ГСМ.
3. **Разделение "грязных" и "чистых" данных** — это не просто техническое решение, это бизнес-правило. Бухгалтер не должен заниматься выверкой — это задача системы и менеджера.
4. **Расширяемость через JSONB** — новые типы документов (зарплата, аренда, логистика) не требуют миграции схемы. Это критично для бизнеса, который растёт и добавляет направления.

1.3. Четыре принципиальные коррекции

Коррекция 1: Semantic API — недостаточно. Нужен Document Graph

Промпт предлагает Semantic API для AI-агентов (find_shift_reports, get_shift_summary). Это хорошо для первой версии, но **AI 2025-2026 года уже умеет работать напрямую с SQL и JSONB**.

Вместо жёстких endpoints предлагаю:

```
Уровень 1: SQL-доступ через read-only реплику (AI пишет запросы сам)
Уровень 2: Semantic API (предвычисленные агрегаты для типовых задач)
Уровень 3: Document Graph (связи между документами)
```

Document Graph — это граф связей между всеми документами:

```
ТТН-1234 ──поступление──→ Резервуар-1 ──хранение──→ Смена-6890
                                                      │
                                          ┌──────────────────┤
                                          ↓                  ↓
                                   Чек-001...Чек-150    Сменный отчёт
                                          │                  │
                                          ↓                  ↓
```

```
                                Банк.реестр        Отчёт о розн. продажах
                                                          |
                                                          ↓
                                                     XML → 1C
```

AI-агент, имея граф, может:

- Найти **причину** расхождения (а не просто факт)
- Предложить **корректировку** с обоснованием
- Построить **цепочку доказательств** для аудитора

Реализация: таблица `document_links` (source_id, target_id, link_type, metadata).

Коррекция 2: Не "ReconciliationEngine" — а Rule Engine с самообучением

Текущая архитектура: набор захардкоженных проверок (cash_vs_receipts, bank_vs_acquiring и т.д.).

Проблема: при добавлении нового типа учёта (например, зарплата) нужно писать новые проверки с нуля.

Предлагаю: Декларативный Rule Engine:

```
# Правила сверки — это данные, а не код
RECONCILIATION_RULES = [
    {
        "id": "cash_vs_receipts",
        "name": "Касса ККТ = сумма чеков (наличные)",
        "doc_type": "shift_report",
        "source_path": "metadata.payments.cash",
        "target": {
            "doc_type": "receipt",
            "aggregate": "SUM(metadata.amount) WHERE metadata.payment_type =
'cash'"
        },
        "thresholds": {
            "ok": 0,            # точное совпадение
            "warning": 100,      # до 100₽ — предупреждение
            "critical": 500      # от 500₽ — критичное
        },
        "auto_correct": {
            "enabled": False,    # пока ручная корректировка
            "ai_suggest": True   # но AI предлагает исправление
        }
```

```
    },
    # ... другие правила
]
```

Самообучение: AI анализирует историю корректировок и:

- Предлагает новые правила (обнаружил паттерн → "всегда расхождение по ДТ в ночную смену на АЗС-208")
- Корректирует пороги (warning/critical) на основе статистики
- Автоматически применяет корректировки с высокой уверенностью (>95%)

Коррекция 3: Слой 3 (Clean) — не просто чистовые документы, а Accounting Kernel

Текущее описание слоя 3 — это генератор XML для 1С. Это слишком узко.

Слой 3 должен быть **ядром учёта**, которое:

- Ведёт **план счетов** (не зависит от 1С!)
- Формирует **проводки** по правилам (настраиваемый маппинг)
- Генерирует **любой** выходной формат (1С XML, SAP IDoc, Excel, API)
- Поддерживает **несколько учётных политик** (бухгалтерский + управленческий + налоговый)

```
Слой 2 (Verified) → Accounting Kernel → 1С XML
                                      → Управленческий отчёт
                                      → Налоговый регистр
                                      → Excel сводная
                                      → AI Summary
```

Это критично для масштабирования на другие типы учёта. Зарплата, аренда, логистика — все проходят через один Accounting Kernel с разными маппингами.

Коррекция 4: Event-Driven вместо Request-Response

Текущая архитектура подразумевает линейный pipeline: загрузка → парсинг → сверка → верификация → экспорт.

Для production-системы с 100+ АЗС нужна **событийная архитектура:**

```
[STS API] ──webhook──> [Event Bus] ──> [Parser Worker]
[Bank CSV] ──upload──> [Event Bus] ──> [Reconciliation Worker]
[1C Feedback] ──xml──> [Event Bus] ──> [Feedback Worker]
```

```
Event Bus = PostgreSQL LISTEN/NOTIFY (простой вариант)
          = Redis Streams (средний)
          = RabbitMQ/Kafka (масштабный)
```

Почему важно:

- Параллельная обработка (100 смен одновременно)
- Устойчивость к сбоям (retry, dead letter queue)
- AI-агент может подписаться на события ("уведоми меня о критических расхождениях")
- Каждый шаг — независимый worker, можно масштабировать отдельно

1.4. Сравнение с альтернативами

Подход	Плюсы	Минусы	Вердикт
Ваш (3 слоя + JSONB)	Гибкость, AI-ready, расширяемость	Требует дисциплины с JSONB	**Правильный выбор**
Всё в 1С	Бухгалтеры знают	Нет API для AI, сложно масштабировать	Не подходит
Чистый NoSQL (MongoDB)	Максимальная гибкость	Нет транзакций, сложные JOIN-ы	Избыточно
Чистый SQL (без JSONB)	Строгая схема	Миграции при каждом новом типе	Слишком жёстко
ERP (SAP/Oracle)	Промышленный	Дорого, сложно, нет AI-first	Не ваш масштаб

---

2. АРХИТЕКТУРНЫЕ РЕШЕНИЯ

2.1. Гибридная модель: когда колонка, когда JSONB

Правило: Если поле участвует в WHERE, JOIN, GROUP BY, ORDER BY — это колонка. Всё остальное — JSONB.

```
┌──────────────────────────────────────────────────────────────┐
│                    КОЛОНКИ (индексируемые)                   │
│  id, doc_type, doc_date, station_id, counterparty_id,        │
│  status, reconciliation_status, created_at, verified_at      │
├──────────────────────────────────────────────────────────────┤
│                    JSONB metadata                            │
```

```
│  Всё специфичное для конкретного типа документа:        │
│  shift_number, operator, sales_by_fuel, payments,       │
│  tank_balances, reconciliation details, ...              │
├──────────────────────────────────────────────────────────┤
│              GENERATED COLUMNS (из JSONB)                 │
│  Часто запрашиваемые поля из metadata:                  │
│  total_amount, operator_name, shift_number               │
│  (вычисляются автоматически, индексируются)              │
└──────────────────────────────────────────────────────────┘
```

Критерий миграции из JSONB → колонку:

- Поле используется в >30% запросов → вынести в колонку
- Поле участвует в сверке → вынести в колонку
- Поле нужно для отчётности → Generated Column

2.2. Индексация

```
-- B-tree: для точных совпадений и диапазонов
CREATE INDEX idx_docs_type_date ON documents (doc_type, doc_date);
CREATE INDEX idx_docs_station_date ON documents (station_id, doc_date);
CREATE INDEX idx_docs_status ON documents (status) WHERE status != 'verified';

-- GIN: для поиска ВНУТРИ JSONB
CREATE INDEX idx_docs_metadata ON documents USING GIN (metadata
jsonb_path_ops);
-- jsonb_path_ops быстрее стандартного GIN для операторов @>, но не
поддерживает ?/? |

-- Частичные GIN-индексы (экономят место):
CREATE INDEX idx_shift_reports_metadata
  ON documents USING GIN (metadata jsonb_path_ops)
  WHERE doc_type = 'shift_report';

-- Generated columns для частых запросов:
ALTER TABLE documents ADD COLUMN total_amount NUMERIC
  GENERATED ALWAYS AS ((metadata->>'total_amount')::numeric) STORED;
CREATE INDEX idx_docs_total_amount ON documents (total_amount);
```

Рекомендация по производительности:

- GIN-индексы тяжелы при записи (замедляют INSERT на 20-40%)
- Для таблицы raw_data GIN НЕ нужен (данные пишутся, не ищутся)
- Для documents — GIN оправдан (читаются чаще, чем пишутся)

2.3. Партиционирование

При 100+ АЗС × 2 смены/день × 365 дней = ~73 000 документов/год на слой 2.

```
-- Партиционирование по месяцам
CREATE TABLE documents (
    id UUID NOT NULL,
    doc_date TIMESTAMPTZ NOT NULL,
    doc_type TEXT NOT NULL,
    -- ...
) PARTITION BY RANGE (doc_date);

-- Автоматическое создание партиций (pg_partman или вручную):
CREATE TABLE documents_2026_01 PARTITION OF documents
    FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE documents_2026_02 PARTITION OF documents
    FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');

-- Партиция raw_data отдельно (растёт быстрее):
CREATE TABLE raw_data (
    id UUID NOT NULL,
    received_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- ...
) PARTITION BY RANGE (received_at);
```

73K docs/year — это немного для PostgreSQL. Партиционирование оправдано, но не критично. Реальная польза — при архивации старых данных (DETACH PARTITION → archive).

2.4. Валидация JSONB

Двухуровневая стратегия:

```
Уровень 1: CHECK constraint (минимальная защита от мусора)
Уровень 2: Pydantic model (полная валидация в Python)
```

```
-- Уровень 1: базовый CHECK
ALTER TABLE documents ADD CONSTRAINT check_metadata_not_null
    CHECK (metadata IS NOT NULL AND metadata != '{}'::jsonb);

-- Для конкретных типов:
ALTER TABLE documents ADD CONSTRAINT check_shift_report_fields
    CHECK (
        doc_type != 'shift_report'
```

```
        OR (
            metadata ? 'shift_number'
            AND metadata ? 'payments'
            AND metadata ? 'sales_by_fuel'
        )
    );
```

```python
# Уровень 2: полная валидация в Python
from pydantic import BaseModel, validator

class ShiftReportMetadata(BaseModel):
    shift_number: int
    shift_start: datetime
    shift_end: datetime
    operator: str
    sales_by_fuel: dict[str, FuelSalesItem]
    payments: PaymentsSummary
    tank_balances: list[TankBalance]

    @validator('shift_end')
    def end_after_start(cls, v, values):
        if v <= values.get('shift_start', v):
            raise ValueError('shift_end must be after shift_start')
        return v
```

---

3. БИЗНЕС-ЛОГИКА И СВЕРКИ

3.1. Матрица сверок (расширенная)

Ваша матрица из FUEL_ACCOUNTING_SYSTEM.md хорошая. Добавляю классификацию:

Сверка	Тип	Автоматизация	Порог warning	Порог critical
Операции ↔ Сменный отчёт (литры)	Точная	100% авто	>0.1%	>1%
Касса ↔ Сменный отчёт (₽)	Точная	100% авто	>0 ₽	>100 ₽
Банк ↔ ККТ безнал	Точная	100% авто	>0 ₽	>100 ₽
Топл.карты ↔ Операции	Поштучная	95% авто	1 непарная	>3 непарных

Сверка	Тип	Автоматизация	Порог warning	Порог critical
Онлайн ↔ Операции	Поштучная	95% авто	1 непарная	>3 непарных
ТТН ↔ Факт приёмки	Нормативная	100% авто	>0.3% (масса)	>0.5%
Остатки резервуаров	Расчётная	100% авто	>0.3%	>0.5%

Ключевое: Пороги — это **конфигурация, а не код**. Хранить в таблице `reconciliation_thresholds` или в справочнике Supabase. Менеджер может менять пороги без релиза.

3.2. Workflow корректировок

```
Расхождение обнаружено
        │
        ▼
┌─ AI-агент анализирует ──────────────────────────┐
│  • Это типовое расхождение? (паттерн в истории)     │
│  • Какова вероятная причина?                         │
│  • Предлагаемая корректировка + уверенность          │
└──────────────────────────────────────────────────┘
        │
        ├── Уверенность > 95% → Автокорректировка + лог
        │                       (менеджер видит в ленте)
        │
        ├── Уверенность 70-95% → Предложение менеджеру
        │                        (approve/reject одним кликом)
        │
        └── Уверенность < 70% → Ручной разбор
                                (менеджер исследует)
```

Типичные паттерны автокорректировки (для обучения AI):

- Возврат по карте: сумма минус → парная операция с тем же чеком
- Тестовая транзакция: 1 литр / 1₽ → исключить из сверки
- Ночная смена пересечение: операция в 00:01 → отнести к предыдущей смене
- Потери в норме: отклонение < 0.3% → списание по нормативу

3.3. Маппинг: документ → проводки

Маппинг должен быть **конфигурируемым**, а не захардкоженным:

```
# Таблица accounting_mappings (в БД или YAML)
POSTING_RULES = {
    "retail_sales_report": {
        "description": "Отчёт о розничных продажах",
        "entries": [
            {
                "condition": "payment_type == 'cash'",
                "debit": "50.01",       # Касса
                "credit": "90.01.1",    # Выручка
                "amount": "line.amount",
                "analytics": {
                    "debit": {"Касса": "Основная касса"},
                    "credit": {"Номенклатура": "line.product"}
                }
            },
            {
                "condition": "payment_type == 'card'",
                "debit": "57.03",       # Транзит (эквайринг)
                "credit": "90.01.1",    # Выручка
                "amount": "line.amount"
            },
            {
                "condition": "payment_type == 'fuel_card'",
                "debit": "62.01",       # Дебиторка
                "credit": "90.01.1",    # Выручка
                "amount": "line.amount",
                "analytics": {
                    "debit": {"Контрагент": "line.counterparty"}
                }
            },
            {
                # Себестоимость
                "debit": "90.02.1",     # Себестоимость
                "credit": "41.01",      # Товары
                "amount": "line.cost_amount",
                "analytics": {
                    "credit": {"Номенклатура": "line.product", "Склад":
"station.name"}
                }
            },
            {
                # НДС
                "debit": "90.03",       # НДС
                "credit": "68.02",      # Расчёты по НДС
                "amount": "line.vat_amount"
            }
```

```
        ]
    },
    "supplier_invoice": {
        "description": "Приходная накладная (ТТН)",
        "entries": [
            {
                "debit": "41.01",       # Товары
                "credit": "60.01",      # Поставщик
                "amount": "line.amount_without_vat"
            },
            {
                "debit": "19.03",       # НДС входящий
                "credit": "60.01",
                "amount": "line.vat_amount"
            }
        ]
    },
    "loss_writeoff_normal": {
        "description": "Списание потерь (в пределах нормы)",
        "entries": [
            {
                "debit": "44.01",       # Расходы на продажу
                "credit": "41.01",
                "amount": "line.amount"
            }
        ]
    },
    "loss_writeoff_excess": {
        "description": "Списание потерь (сверх нормы)",
        "entries": [
            {
                "debit": "94",          # Недостачи и потери
                "credit": "41.01",
                "amount": "line.amount"
            }
        ]
    },
    "cash_collection": {
        "description": "Инкассация",
        "entries": [
            {
                "debit": "51",          # Расчётный счёт
                "credit": "50.01",      # Касса
                "amount": "doc.amount"
            }
        ]
```

```
    }
}
```

---

4. ИНТЕГРАЦИЯ С 1С

4.1. Выбор формата обмена

Формат	Для чего подходит	Ограничения
CommerceML 2.x	Каталоги товаров, простые заказы	НЕ поддерживает проводки, субконто
EnterpriseData	Полный обмен (1С:Предприятие 8.3+)	Сложный, жёсткая привязка к версии
Custom XML	Любая структура	Нужна обработка на стороне 1С
HTTP API (OData)	Прямое создание документов	Нужна публикация базы 1С

Рекомендация: Custom XML + обработка загрузки в 1С.

Причины:

- CommerceML НЕ поддерживает проводки (Дт/Кт) — а вам нужно передавать готовые проводки
- EnterpriseData слишком привязан к конкретной конфигурации 1С
- HTTP API (OData) — хорош, но требует публикации базы 1С в сеть
- Custom XML — максимальная гибкость, обработку пишем один раз

4.2. Структура Custom XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<TradeFrame version="1.0" export_date="2026-02-24T20:00:00">

  <Document type="RetailSalesReport" id="uuid-doc-1">
    <Header>
      <Number>РПР-6890</Number>
      <Date>2026-02-24</Date>
      <Station code="208" name="АЗС 208"/>
      <Shift number="6890"/>
      <Period from="2026-02-24T07:31:00" to="2026-02-24T19:45:00"/>
      <TotalAmount>154646.00</TotalAmount>
```

```
    <Currency>RUB</Currency>
  </Header>

  <Lines>
    <Line num="1">
      <Product code="АИ-92" name="Бензин АИ-92"/>
      <Quantity unit="л">1240.000</Quantity>
      <Price>52.50</Price>
      <Amount>65100.00</Amount>
      <VATRate>20</VATRate>
      <VATAmount>10850.00</VATAmount>
    </Line>
    <Line num="2">
      <Product code="АИ-95" name="Бензин АИ-95"/>
      <Quantity unit="л">890.000</Quantity>
      <Price>56.80</Price>
      <Amount>50552.00</Amount>
      <VATRate>20</VATRate>
      <VATAmount>8425.33</VATAmount>
    </Line>
    <Line num="3">
      <Product code="ДТ" name="Дизельное топливо"/>
      <Quantity unit="л">670.000</Quantity>
      <Price>58.20</Price>
      <Amount>38994.00</Amount>
      <VATRate>20</VATRate>
      <VATAmount>6499.00</VATAmount>
    </Line>
  </Lines>

  <Entries>
    <Entry num="1" description="Выручка наличные">
      <Debit account="50.01" analytics='{"Касса": "Основная"}'/>
      <Credit account="90.01.1" analytics='{"Номенклатура": "АИ-92"}'/>
      <Amount>42300.00</Amount>
    </Entry>
    <Entry num="2" description="Выручка эквайринг">
      <Debit account="57.03"/>
      <Credit account="90.01.1"/>
      <Amount>78200.00</Amount>
    </Entry>
    <Entry num="3" description="Выручка топливные карты">
      <Debit account="62.01" analytics='{"Контрагент": "ООО Логистик"}'/>
      <Credit account="90.01.1"/>
      <Amount>24146.00</Amount>
    </Entry>
```

```
      <Entry num="4" description="Выручка онлайн">
        <Debit account="57.03"/>
        <Credit account="90.01.1"/>
        <Amount>10000.00</Amount>
      </Entry>
    </Entries>
  </Document>

</TradeFrame>
```

4.3. Механизм обмена

```
TradeFrame                          1С:Бухгалтерия
    │                                     │
    ├──export_xml──→ /shared/to_1c/*.xml   │
    │                       │              │
    │                       └──→ Обработка загрузки
    │                               │
    │                               ├── Создать документ
    │                               ├── Заполнить строки
    │                               ├── Записать проводки
    │                               ├── Провести
    │                               │
    │                               └──→ /shared/from_1c/feedback.xml
    │                                           │
    ├──import_feedback──────────────────────────┘
    │
    └── Обновить status: posted / error
```

Feedback XML:

```
<TradeFrameFeedback date="2026-02-24T20:15:00">
  <Result doc_id="uuid-doc-1">
    <Status>posted</Status>
    <OneC_GUID>e1cib-guid-from-1c</OneC_GUID>
    <OneC_Number>РПР-000123</OneC_Number>
    <PostedAt>2026-02-24T20:14:55</PostedAt>
  </Result>
  <Result doc_id="uuid-doc-2">
    <Status>error</Status>
    <ErrorCode>NomenclatureNotFound</ErrorCode>
    <ErrorMessage>Номенклатура "АИ-92 Евро" не найдена в
справочнике</ErrorMessage>
```

```
  </Result>
</TradeFrameFeedback>
```

4.4. Откат (Rollback)

Правило: 1С НЕ правится вручную. Если ошибка:

```
1. Получили error из 1С → status = 'error' в слое 3
2. Диагностика: AI или менеджер определяет причину
3. Исправление в слое 2 (корректировка документа)
4. Повторная верификация
5. Генерация нового чистового документа (слой 3)
6. Повторный экспорт

Если документ уже проведён в 1С и нужно отменить:
1. Отправляем команду "отменить проведение" через XML
2. 1С отменяет проведение, подтверждает в feedback
3. Экспортируем исправленный документ
4. 1С проводит новый документ
```

---

5. AI-АГЕНТЫ: АРХИТЕКТУРА ИНТЕГРАЦИИ

5.1. Три уровня AI-участия

```
Уровень 1: НАБЛЮДАТЕЛЬ (read-only)
──────────────────────────────────

• Аналитика: "покажи смены с расхождениями за февраль"
• Отчёты: "маржа по видам топлива за квартал"
• Аномалии: "необычные паттерны продаж"
• Инструменты: SQL-доступ, Semantic API

Уровень 2: СОВЕТНИК (предлагает, человек подтверждает)
──────────────────────────────────────────────────────

• Корректировки: "расхождение 50₽ по кассе — вероятно, забыли пробить чек"
• Оптимизация: "рекомендую заказать ДТ на АЗС-208, остатков на 2 дня"
• Пороги: "предлагаю увеличить warning для ДТ до 0.4% — часто ложные"
• Инструменты: Semantic API + CorrectionAPI (draft mode)

Уровень 3: ИСПОЛНИТЕЛЬ (действует автономно, в рамках)
──────────────────────────────────────────────────────

• Автокорректировка типовых расхождений (уверенность >95%)
• Автоматическая верификация "чистых" смен (все сверки ok)
```

```
• Генерация чистовых документов при достижении verified
• Инструменты: Full API access с ограничениями
```

5.2. Semantic API (конкретные endpoints)

```python
# ===== ЧТЕНИЕ =====

@router.get("/api/v1/shifts")
async def list_shifts(
    date_from: date,
    date_to: date,
    station_id: UUID = None,
    has_discrepancy: bool = None,
    status: str = None
) -> list[ShiftSummary]:
    """Список смен с фильтрами. AI использует для поиска."""

@router.get("/api/v1/shifts/{shift_id}")
async def get_shift(shift_id: UUID) -> ShiftDetail:
    """Полная детализация смены. AI использует для анализа."""

@router.get("/api/v1/shifts/{shift_id}/reconciliation")
async def get_reconciliation(shift_id: UUID) -> ReconciliationResult:
    """Результаты всех сверок. AI использует для диагностики."""

@router.get("/api/v1/shifts/{shift_id}/graph")
async def get_document_graph(shift_id: UUID) -> DocumentGraph:
    """Граф связанных документов. AI использует для трейсинга."""

@router.get("/api/v1/analytics/anomalies")
async def detect_anomalies(
    date_from: date,
    date_to: date,
    min_severity: str = "warning"
) -> list[Anomaly]:
    """Обнаруженные аномалии. AI использует для проактивного мониторинга."""

# ===== ДЕЙСТВИЯ =====

@router.post("/api/v1/corrections/suggest")
async def suggest_correction(
    document_id: UUID,
    discrepancy_id: UUID
) -> CorrectionSuggestion:
    """AI предлагает корректировку. Возвращает suggestion с confidence."""
```

```python
@router.post("/api/v1/corrections/apply")
async def apply_correction(
    document_id: UUID,
    field_path: str,
    new_value: Any,
    reason: str,
    auto: bool = False  # True = AI автокорректировка
) -> CorrectionResult:
    """Применить корректировку. auto=True только для confidence >95%."""

@router.post("/api/v1/documents/{doc_id}/verify")
async def verify_document(doc_id: UUID) -> VerificationResult:
    """Верифицировать документ. Проверяет все preconditions."""

# ===== SQL (для продвинутых запросов) =====

@router.post("/api/v1/query")
async def execute_query(
    sql: str,  # read-only SQL
    params: dict = None
) -> QueryResult:
    """
    Выполнить SQL на read-only реплике.
    AI может писать произвольные аналитические запросы.
    Ограничения: только SELECT, timeout 30s, max 10K rows.
    """
```

5.3. Контекст для AI: минимизация токенов

Проблема: Описание всей схемы БД — это тысячи токенов. AI не нужно знать всё.

Решение: Dynamic Schema Description

```python
@router.get("/api/v1/schema/context")
async def get_ai_context(
    task: str  # "reconciliation", "analytics", "correction"
) -> AIContext:
    """
    Возвращает минимальный контекст для AI-агента.

    task="reconciliation":
      - Описание таблицы documents (ключевые поля)
      - Описание reconciliation_checks
      - Примеры metadata для shift_report
      - Доступные API endpoints
```

```
    task="analytics":
      - Описание materialized views
      - Примеры запросов
      - Доступные агрегаты
    """
```

Результат: Вместо 5000 токенов на полную схему — 500-800 токенов на релевантный контекст.

---

6. РАСШИРЯЕМОСТЬ НА ДРУГИЕ ТИПЫ УЧЁТА

6.1. Универсальная модель

Ваша архитектура уже универсальна. Вот как добавить новый тип учёта:

```
Шаг 1: Новый загрузчик (RawDataLoader)
  → class SalaryCSVLoader(RawDataLoader)

Шаг 2: Новый парсер (DocumentParser)
  → class SalarySlipParser(DocumentParser)

Шаг 3: Новые правила сверки (декларативные)
  → salary_rules.yaml

Шаг 4: Новый маппинг проводок (конфигурация)
  → salary_posting_rules.yaml

Шаг 5: Готово. Без изменения ядра системы.
```

6.2. Примеры расширения

Модуль	Загрузчик	Парсер	Сверки	Проводки
ГСМ	STS API, Bank CSV, PDF ТТН	ShiftReport, Receipt, ТТН	7 типов	50/51/62→90, 41→60
Зарплата	1С ЗУП export, табель	PayrollSlip, Timesheet	Табель↔начисление, НДФЛ↔13%	20→70, 70→68

Модуль	Загрузчик	Парсер	Сверки	Проводки
Аренда	Договоры, акты, счета	LeaseContract, Invoice	Акт↔договор, оплата↔счёт	44→76, 76→51
Логистика	GPS, путевые листы, ТТН	WaybillParser, GPSTrack	Маршрут↔план, расход↔норма	10→60, 44→10
Продажи (не ГСМ)	Кассы, e-commerce	SaleReceipt, OnlineOrder	Касса↔банк, склад↔отгрузка	50/62→90, 90→41

6.3. Общие справочники

```
-- Контрагенты (общие для всех модулей)
CREATE TABLE counterparties (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    inn TEXT,
    kpp TEXT,
    type TEXT, -- supplier, customer, employee, bank
    metadata JSONB DEFAULT '{}'
);

-- Номенклатура (общая)
CREATE TABLE products (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    code TEXT,
    unit TEXT, -- л, кг, шт, час
    type TEXT, -- fuel, goods, service, salary_type
    metadata JSONB DEFAULT '{}'
);

-- Станции / подразделения (общие)
CREATE TABLE locations (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT, -- station, office, warehouse
    metadata JSONB DEFAULT '{}'
);
```

---

7. ПЛАН РЕАЛИЗАЦИИ

7.1. MVP (4-6 недель)

Цель: Одна АЗС, один тип документа (сменный отчёт), сквозной pipeline RAW → 1С.

Неделя	Что делаем
1	БД: таблицы raw_data, documents, reconciliation_checks. Загрузчик STS API
2	Парсер сменного отчёта. Базовые сверки (касса, банк, остатки)
3	API корректировок. Статусная модель (draft→verified). UI: список смен + детализация
4	Генерация чистовых документов (слой 3). XML экспорт в 1С
5	Обработка feedback из 1С. Базовый AI-агент (аналитика read-only)
6	Тестирование на реальных данных. Исправление багов

7.2. v1.0 (ещё 4-6 недель)

Компонент	Что добавляем
Загрузчики	Bank CSV, PDF ТТН (с OCR), Excel прайсы
Сверки	Все 7 типов из матрицы. Rule Engine (декларативный)
AI	Предложение корректировок. Автокорректировка типовых
UI	Дашборд здоровья. Протокол сверки. Граф документов
Масштаб	10+ АЗС. Batch processing. Event-driven pipeline

7.3. v2.0 (ещё 6-8 недель)

Компонент	Что добавляем
Accounting Kernel	План счетов. Конфигурируемые маппинги. Мультиорганизация
AI	Самообучение (пороги, паттерны). Аномалии. Прогнозы
Модули	Второй тип учёта (аренда или зарплата)
Масштаб	100+ АЗС. Партиционирование. Redis кэш

7.4. Риски и митигации

Риск	Вероятность	Митигация
JSONB-хаос (нет дисциплины)	Высокая	Pydantic-валидация, code review, тесты
1С-интеграция ломается при обновлении	Средняя	Версионирование XML, тесты на sandbox 1С
AI делает некорректные корректировки	Средняя	Порог уверенности, аудит, ролбэк
Производительность GIN-индексов	Низкая	Мониторинг, partial indexes
OCR ТТН — низкое качество	Высокая	Fallback на ручной ввод, обучение модели

---

8. ОТВЕТЫ НА ВОПРОСЫ ИЗ ПРОМПТА

8.1. Типы первичных документов (помимо смен, ТТН, топливных карт)

- Акты сверки с контрагентами (поставщики топлива)
- Акты инвентаризации (остатки резервуаров)
- Калибровочные протоколы (точность ТРК)
- Претензии покупателей (возвраты)
- Путевые листы (если есть собственная логистика)
- Табели учёта рабочего времени (операторы АЗС)

8.2. Формат интеграции с 1С

Рекомендация: файловый обмен (XML в папку) для MVP, HTTP API для v2.0.

Файловый обмен надёжнее, не требует публикации базы 1С, работает даже при сбое сети. HTTP API добавить позже для real-time синхронизации.

8.3. Роли пользователей

Действие	Кто
Первичка загружается	Автоматически (STS API, webhooks)
Сверяет и корректирует	Менеджер + AI-агент

Действие	Кто
Подтверждает (verify)	Менеджер или старший менеджер
Экспортирует в 1С	Бухгалтер (кнопка "Выгрузить")

8.4. Производительность

- **100 АЗС × 2 смены = 200 документов/день** (основные)
- **+ транзакции: ~50K/день** (детализация)
- **Время RAW → verified:** 1-5 минут (автоматически), до 24 часов (с ручной корректировкой)
- **SLA выгрузки в 1С:** не позднее конца следующего рабочего дня

8.5. AI-агент: приоритеты

1. **Обнаружение аномалий** — критические расхождения в реальном времени
2. **Предложение корректировок** — экономит 80% времени менеджера
3. **Аналитика по запросу** — "сравни потери ДТ за январь и февраль"
4. **Автокорректировка типовых** — только после накопления истории (v1.0+)
5. **Прогнозирование** — заказ топлива, планирование инкассации (v2.0)

8.6. Следующий модуль после ГСМ

Рекомендую **корпоративные продажи (топливные карты)** — он уже частично реализован в TradeCorp, данные уже интегрированы в сменный отчёт, и это следующий логический шаг (выставление счетов контрагентам по топливным картам).

8.7. Окружение развёртывания

Рекомендую **Docker + VPS** (у вас уже есть spb-3-vm-ipoc). PostgreSQL в Docker с volume mount. FastAPI в Docker. Nginx reverse proxy.

8.8. Тестирование

```
# Тест сверки: создаём "идеальные" данные → все сверки ok
async def test_reconciliation_all_ok():
    raw = create_perfect_shift_raw_data()
    doc = await parse_and_save(raw)
    result = await reconciliation_engine.reconcile(doc.id)
    assert result['status'] == 'ok'
    assert result['critical_count'] == 0

# Тест сверки: создаём расхождение → critical
```

```python
async def test_reconciliation_cash_discrepancy():
    raw = create_shift_with_cash_diff(diff=500)
    doc = await parse_and_save(raw)
    result = await reconciliation_engine.reconcile(doc.id)
    assert result['status'] == 'discrepancy'
    assert any(c['type'] == 'cash_vs_receipts' and c['status'] == 'critical'
               for c in result['checks'])

# Тест экспорта: verified doc → valid XML
async def test_export_produces_valid_xml():
    doc = create_verified_document()
    xml = await exporter.export(doc.id)
    schema = lxml.etree.XMLSchema(lxml.etree.parse('schema.xsd'))
    assert schema.validate(lxml.etree.fromstring(xml))
```

---

9. ИТОГОВЫЕ РЕКОМЕНДАЦИИ

Что в промпте хорошо и нужно сохранить:

1. **3-слойная архитектура** — правильный фундамент
2. **Иммутабельность RAW** — критично для аудита
3. **"1С = принтер"** — верная философия
4. **JSONB для metadata** — правильный выбор для гибкости
5. **Статусная модель** — чёткий workflow
6. **Audit trail корректировок** — обязательно для учёта

Что нужно добавить/изменить:

1. **Document Graph** — связи между документами (вместо плоского списка)
2. **Декларативный Rule Engine** — правила сверок = данные, а не код
3. **Accounting Kernel** — слой 3 должен быть ядром учёта, а не просто XML-генератором
4. **Event-Driven архитектура** — для масштабирования и AI-подписок
5. **Dynamic Schema Context** — минимизация токенов для AI
6. **Конфигурируемые маппинги проводок** — не хардкодить Дт/Кт
7. **3 уровня AI-участия** — наблюдатель → советник → исполнитель
8. **Custom XML** для 1С (не CommerceML) — нужны проводки

Технологический стек (финальный):

Компонент	Технология	Обоснование
API	FastAPI + asyncpg	Async, быстрый, типизированный
БД	PostgreSQL 16 + JSONB	Гибрид: надёжность SQL + гибкость JSON
Очереди	PostgreSQL LISTEN/NOTIFY (MVP), Redis Streams (v1)	Простота → масштаб
Кэш	Redis	Сводные, сессии, rate limiting
AI	Claude API (function calling)	Лучше для структурированных задач
XML	lxml	Быстрее ElementTree в 5-10×
Валидация	Pydantic v2	Строгая типизация metadata
Тесты	pytest + pytest-asyncio + testcontainers	Реальный PostgreSQL в тестах
Деплой	Docker Compose на VPS	Уже есть инфраструктура

---

10. КРИТИЧЕСКОЕ УТОЧНЕНИЕ: ПРОВОДКИ В 1С

Исследование интеграции с 1С показало важный нюанс:

Не передавать готовые проводки (Дт/Кт) в 1С

Антипаттерн (как описано в промпте):

```
<!-- НЕ НАДО: готовые проводки в XML -->
<Проводка>
  <Дебет>50.01</Дебет>
  <Кредит>90.01.1</Кредит>
  <Сумма>42300.00</Сумма>
</Проводка>
```

Правильный подход:

```
<!-- ПРАВИЛЬНО: передать документ, 1С сама сделает проводки -->
<DocumentObject.ОтчётОРозничныхПродажах>
  <Товары>
    <Row>
      <Номенклатура>Бензин АИ-92-К5</Номенклатура>
```

```
      <Количество>2459.80</Количество>
      <Цена>54.00</Цена>
      <Сумма>132829.20</Сумма>
      <СтавкаНДС>НДС20</СтавкаНДС>
    </Row>
  </Товары>
  <Оплата>
    <Row><ВидОплаты>Наличные</ВидОплаты><Сумма>85400.00</Сумма></Row>
    <Row><ВидОплаты>Платёжная карта</ВидОплаты><Сумма>142300.00</Сумма></Row>
  </Оплата>
</DocumentObject.ОтчётОРозничныхПродажах>
```

Почему:

- 1С имеет свою учётную политику и автоматически формирует проводки при проведении документов
- Счета учёта в БП 3.0 хранятся в регистре "СчетаУчётаНоменклатуры" — 1С подставляет их сама
- Передача проводок напрямую ломает целостность учёта в 1С
- Единственное исключение: документ "Операция" для ручных корректировок

Как это влияет на архитектуру:

Accounting Kernel (слой 3) по-прежнему формирует проводки, но использует их **для управленческого учёта и контроля**, а не для передачи в 1С:

```
Слой 2 (verified) → Accounting Kernel
                     ├── Проводки → для управленческих отчётов TradeFrame
                     ├── Проводки → для контроля (сравнить с 1С после
проведения)
                     └── XML документ (без проводок) → 1С:БП
                                                        └── 1С сама формирует
проводки
                                                        └── Feedback →
сравниваем
```

Рекомендуемый формат обмена с 1С

Аспект	Решение
Основной транспорт	Файловый обмен (XML в папку) — надёжный, работает при выключенной 1С

Аспект	Решение
Дополнительно	HTTP-сервис 1С — для справочников, статусов, оперативных запросов
Формат XML	Custom (не CommerceML) — CommerceML не покрывает специфику ГСМ
Проводки	НЕ передавать, 1С формирует сама при проведении
Обратная связь	Файл квитанции: статус, номер 1С, ошибки
Идемпотентность	Уникальный ИдСообщения в каждом XML
Справочники	Mapping по кодам, НЕ по наименованиям

Типичные ловушки интеграции с 1С

1. **Справочники:** Использовать коды (AI92K5), не наименования ("Бензин АИ-92") — наименования могут отличаться
2. **Округление НДС:** Считать построчно, итог = сумма строк (не наоборот)
3. **Кодировка:** Всегда UTF-8 с BOM для файлового обмена
4. **Нумерация:** Передавать как "внешний номер", 1С присваивает свой
5. **Закрытый период:** Проверять дату закрытия перед отправкой документа
6. **Пересчёт литры/кг:** Хранить ОБА значения + плотность + температуру

---

11. ОТВЕТ НА ГЛАВНЫЙ ВОПРОС: AI-Native учёт в бизнесе

Почему трёхслойная архитектура — это правильный фундамент для будущего

Ваш вопрос: *"Правильный ли это подход с учётом появления AI-агентов к учёту в бизнесе?"*

Да, и вот почему это не просто "хороший подход", а единственный правильный:

1. AI нужна структура, не интерфейс

Традиционный учёт:

```
Человек → UI (кнопки, формы) → Данные → Отчёты
```

AI-native учёт:

```
Данные → API → AI-агент → Данные → Человек (контроль) → Отчёты
```

Ваша архитектура убирает "UI-зависимость" и даёт AI прямой доступ к данным через API. Это фундаментально правильно.

2. JSONB = самодокументируемость для AI

Реляционная схема с 50 таблицами и 200 FK — кошмар для AI (нужен огромный контекст).
JSONB metadata в одной таблице documents — AI читает структуру прямо из данных:

```
{"shift_number": 6890, "payments": {"cash": 42300, "card": 78200}}
```

AI сразу понимает, что это за данные, без описания схемы.

3. Декларативные правила = AI может их создавать

Если сверки — это код (Python-функции), AI может их только читать.
Если сверки — это конфигурация (YAML/JSON), AI может их **создавать и модифицировать**:

```
AI: "Я заметил, что на АЗС-208 расхождение по ДТ всегда в ночную смену.
     Предлагаю добавить правило: если расхождение <0.2% + ночная смена + ДТ →
auto_ok"
```

4. Document Graph = AI рассуждает о причинах

Без графа: "Расхождение 500₽ по кассе" — AI может только сообщить.
С графом: "Расхождение 500₽ → чек №142 был отменён → возврат не прошёл через банк → нужно повторить refund" — AI **находит причину и предлагает действие**.

5. Иммутабельность = AI может учиться на истории

Append-only RAW данные + полный audit trail корректировок = идеальный тренировочный набор для AI:

- "Какие расхождения обычно оказываются ложными?"
- "Какие корректировки менеджер делает чаще всего?"
- "Есть ли сезонные паттерны потерь?"

Что делает эту архитектуру по-настоящему AI-native

Свойство	Классический учёт	Ваша архитектура	Почему важно для AI
Данные	Спрятаны в UI	Доступны через API	AI работает с API
Схема	Жёсткая (50+ таблиц)	Гибкая (JSONB)	AI читает данные без описания
Правила	Захардкожены	Декларативные	AI может создавать/менять
Связи	Неявные (FK)	Явный граф	AI рассуждает о причинах
История	Перезаписывается	Иммутабельная	AI учится на истории
Действия	Только человек	AI с порогом уверенности	Автоматизация 80% рутины

Это подход к ЛЮБОМУ учёту, не только ГСМ

```
ГСМ:      STS API → RAW → Documents → Reconciliation → Clean → 1C
Зарплата: 1С ЗУП  → RAW → Documents → Reconciliation → Clean → 1C
Аренда:   Договор → RAW → Documents → Reconciliation → Clean → 1C
Продажи:  Касса   → RAW → Documents → Reconciliation → Clean → 1C
```

Одна архитектура, разные загрузчики + парсеры + правила сверки. AI-агент один на все модули — он понимает JSONB metadata любого типа документа.

Куда это ведёт через 2-3 года

```
Сегодня:   AI предлагает → Человек подтверждает → Система исполняет
Через год:  AI исполняет типовое → Человек проверяет исключения
Через 2-3:  AI ведёт полный учёт → Человек задаёт вопросы и утверждает
отчётность
```

Ваша архитектура готова ко всем трём этапам. Это не переделка — это постепенное увеличение автономности AI-агента через конфигурацию (пороги уверенности, разрешённые действия)